

TicketMesh — Universal Configurable Ticketing & Reservation Platform

FULL 53-PART MASTER DEVELOPMENT DOCUMENT

01. Product Vision & Scope

Universal, configurable, multi-provider ticketing and reservation platform covering bus, train, movie, events, sports, flight, ferry, attractions and future domains.

02. Global Benchmark & Product Positioning

Benchmark the aggregation model of Omio-like travel platforms and the inventory/event model of Ticketmaster-like platforms. TicketMesh differentiates through domain-agnostic configuration, provider adapters, AI-native workflows and analytics.

03. Universal Ticketing Model

Use a canonical TicketProduct, TicketType, Provider, Inventory, Offer, Reservation, Booking, Ticket and Policy model. Domain behavior is represented by capabilities and configuration.

04. Multi-Domain Support

Bus: routes, boarding/drop-off, luggage, seats. Train: stations, coaches, classes, berths, quotas. Movie: cinema, screen, show, seat category, food add-ons. Events/sports: venue, sections, rows, seats, general admission, VIP and season tickets.

05. Multi-Provider Aggregation

Search multiple providers in parallel, normalize responses, deduplicate equivalent offers, compare price/time/availability/policies and rank results.

06. Provider Adapter Framework

Canonical ProviderAdapter operations: search, availability, seat map, price, hold, release, book, confirm, get booking, cancel, refund, modify, ticket retrieval and validation.

07. Provider Onboarding

Admin-configured provider records with provider code, country, currency, timezone, API endpoint, auth mode, credentials reference, capabilities, rate limits, timeout, retries, circuit breaker, webhook/polling and reconciliation.

08. Capability System

Providers declare supported capabilities such as search, real-time inventory, seat maps, hold, booking, cancellation, refund, modification, QR tickets and webhooks. Unsupported operations are handled explicitly.

09. Configuration-Driven Product Engine

Configure ticket types, workflows, fields, passenger models, seat selection, policies, fees, taxes, promotions, notifications and validation without changing core code.

10. Universal Search

Search service parses filters and queries providers concurrently, normalizes offers, validates availability, calculates total price and returns a common UniversalOffer model.

11. AI Conversational Search

Natural language such as 'two seats to Kandy tomorrow morning' becomes structured intent and calls authorized search tools. AI must not invent availability.

12. Multi-Ticket / Multi-Service Trip

Represent a Trip containing multiple Booking and Journey items, enabling train + bus + movie + event combinations and one unified itinerary.

13. Smart Trip Planner

AI plans transport and activities based on time, budget, preferences and constraints, then produces an itinerary with bookable options.

14. Inventory Management

Maintain AVAILABLE → HELD → CONFIRMED and HELD → EXPIRED → AVAILABLE. Support seat inventory, capacity inventory, general admission and provider synchronization.

15. Reservation & Concurrency

Use idempotency keys, optimistic locking, reservation expiry, transactional boundaries, distributed coordination where necessary, provider transaction IDs and reconciliation.

16. Booking Engine

Create a provider-independent booking workflow with validation, reservation, payment authorization, provider confirmation, ticket issuance and compensating actions for partial failures.

17. Payment Abstraction

Provider-independent payment interface supporting cards, wallets, bank methods, refunds, webhooks, idempotency and country/currency-specific routing. Use sandbox or mock providers for free development.

18. Refund & Cancellation

Policy-driven cancellation and refund calculation, provider-specific rules, partial refunds, cancellation windows, refund status tracking and reconciliation.

19. Ticket Issuing & Validation

Generate canonical tickets, QR/barcode payloads, ticket status, validation events and optional provider-issued ticket references. Include offline validation capability for suitable ticket types.

20. Pricing & Fare Engine

Support fixed, tiered, time-based, demand-based, promotional and AI-recommended pricing. Separate base fare, taxes, service fees, discounts and provider commission.

21. Promotion Engine

Rule-based promotions for early bird, weekday, student, group, loyalty, provider campaigns, coupons and targeted offers. Keep rules configurable.

22. Customer & Loyalty

Profiles, saved passengers, booking history, preferences, favorites, wallet/credits, loyalty points, memberships, coupons and consent management.

23. Admin & Operations Console

Provider management, ticket-type configuration, routes/venues, seat maps, pricing, policies, promotions, bookings, refunds, fraud alerts, reconciliation, system health and analytics.

24. No-Code / Low-Code Configuration

Admin can define a new product domain through forms and capability selection. Configuration versions must be auditable and deploy independently from application binaries.

25. Multi-Tenancy SaaS

Tenant isolation for travel companies, cinemas, event organizers, operators and enterprise customers. Apply tenant-aware authorization, data partitioning and configuration.

26. AI Assistant

Conversational support for search, booking status, policy explanation, cancellation eligibility and itinerary questions using RAG plus live platform tools.

27. AI Agent Architecture

Controlled tools for search, availability, reservation, booking, cancellation, refund and analytics. Use least privilege, explicit confirmation for financial actions, tool schemas, audit logs and policy enforcement.

28. AI Recommendation Engine

Rank cheapest, fastest, best value, best cancellation policy, best seat, lowest risk and personalized options using rules plus ML ranking.

29. AI Seat Recommendation

Recommend seats using screen/vehicle geometry, price, seat category, availability, group size and user preferences. Support movie, bus, train and event layouts.

30. AI Price Prediction

Predict future price and demand using historical sales, seasonality, day/time, inventory, event attributes and price history. Expose confidence and avoid presenting predictions as guarantees.

31. Dynamic Pricing Intelligence

Provide operator-side recommended pricing and guardrails. Strategies include fixed, tiered, demand-based and AI-assisted pricing with min/max boundaries and auditability.

32. AI Fraud & Cyber Risk

Detect abnormal booking velocity, bot-like activity, account takeover patterns, payment anomalies, promo abuse, suspicious device/IP behavior and ticket scalping patterns.

33. Disruption & Recovery Intelligence

Detect provider failures, delays, cancellations and inventory mismatches; identify affected bookings, propose alternatives, notify customers and initiate rebooking workflows.

34. Ticket Intelligence / Data Analytics

Business dashboards for revenue, bookings, ticket sales, refunds, cancellations, conversion, AOV, provider performance, inventory utilization and customer behavior.

35. AI Data Analyst

Natural-language analytics over a read-only semantic layer: question → metric mapping → validated SQL → warehouse → statistical analysis → chart/explanation/forecast. Enforce row-level security and query limits.

36. Data Platform

Operational events flow through Kafka into analytical storage. Use PostgreSQL for transactions, Parquet/DuckDB for local/offline analytics, S3 for durable cloud data and an optional warehouse later.

37. ML Platform

Python services for demand forecasting, cancellation prediction, fraud scoring, recommendations and anomaly detection. Track datasets, features, model versions, evaluation metrics and deployment status.

38. Event-Driven Architecture

Publish BookingCreated, ReservationHeld, ReservationExpired, PaymentCompleted, TicketIssued, TicketCancelled, RefundCompleted, InventoryChanged, ProviderFailed and FraudAlert events.

39. Microservices Architecture

Recommended services: gateway, identity, tenant, catalog, ticket-type configuration, provider, provider-integration, search, availability, inventory, reservation, booking, pricing, payment, refund, ticket, notification, promotion, fraud, trip, analytics, recommendation and AI.

40. API Gateway & Edge

Gateway provides routing, authentication, authorization, throttling, request correlation, API versioning, validation and observability. Keep internal services private in Kubernetes.

41. Security Architecture

OAuth2/OIDC, Spring Security, RBAC/ABAC, tenant isolation, encryption, secrets management, audit trails, secure headers, rate limiting, mTLS where appropriate, PII minimization and zero-trust service access.

42. AI Security

Prompt-injection defense, system prompt isolation, PII redaction, tool authorization, output validation, retrieval filtering, data-loss prevention, model allowlists, AI audit logs and adversarial evaluation.

43. DevOps / DevSecOps

Automate build, test, scan, SBOM, containerization, artifact publication, GitOps deployment, DAST, progressive rollout, smoke tests and rollback. Security checks are mandatory gates.

44. CI/CD Pipeline

GitHub Actions: lint/format → compile → unit → ArchUnit → dependency scan → Gitleaks → Checkov → Syft SBOM → container build → Trivy/Grype → Testcontainers → Pact → Playwright → k6 → publish → Argo CD staging → ZAP → approval → Argo Rollouts production → smoke/rollback.

45. Kubernetes & GitOps

Amazon EKS production target; Helm packaging; Argo CD desired-state deployment; Argo Rollouts canary/blue-green; KEDA for Kafka/event workloads; HPA for services; cautious VPA; Network Policies for east-west restrictions.

46. AWS Cloud Architecture

Route 53 at DNS, edge/load balancer/API Gateway at ingress, EKS for services, RDS PostgreSQL, DynamoDB for selected key-value use cases, S3 for object/data lake storage, Lambda for lightweight asynchronous jobs and CloudWatch for AWS-native monitoring.

47. Free / Offline Development

Core MVP must run without paid APIs: Docker/Compose, PostgreSQL, Redis, Kafka, MinIO, DuckDB, Parquet, Prometheus, Grafana, Jaeger and local AI models. Provide mock bus/train/movie providers and deterministic payment simulator.

48. Frontend Web & Mobile Strategy

Use React + TypeScript for the primary web consumer application and admin portal because it is the fastest browser-first choice. If a mobile app is added later, React Native can reuse TypeScript domain/API concepts.

49. Testing & Quality

JUnit, Mockito, ArchUnit, REST Assured, Testcontainers, Pact, Playwright, k6 and JMH. Test concurrency, duplicate callbacks, provider timeout, partial booking, refund failure, inventory mismatch and reconciliation.

50. Observability

OpenTelemetry tracing, Jaeger, Prometheus, Grafana, Micrometer and structured JSON logs. Include traceId, spanId, correlationId, tenantId, providerId and bookingId. Monitor both technical and business SLIs.

51. Repository / Git Format

Monorepo: universal-ticketing-platform/{docs,services,integrations,ai,ml,data,frontend,infrastructure,tests,scripts}. Use main/develop/feature branches or trunk-based protected main, pull requests, CODEOWNERS, conventional commits, semantic versioning and release tags.

52. Project Delivery Roadmap

Phase 1 foundation/local platform; Phase 2 identity/gateway/configuration; Phase 3 provider framework/search/inventory; Phase 4 booking/payment/tickets; Phase 5 bus/train/movie simulators; Phase 6 analytics; Phase 7 RAG/AI assistant/agents; Phase 8 ML/fraud/recommendations; Phase 9 EKS/GitOps/AWS; Phase 10 security, performance, chaos and production hardening.

53. MVP + Definition of Done + Portfolio Evidence

MVP: auth, configurable ticket type, three mock providers, multi-provider search, normalized offers, seat/availability, hold/expiry, booking, mock payment, QR ticket, cancellation/refund, Kafka events, React UI, admin configuration, observability, CI security scans and local AI. Done means code + tests + security + observability + docs + deployment + failure handling. Publish ADRs, diagrams, threat model, benchmarks, AI evaluation, provider failure tests and reproducible setup.

54. Complete Technology Matrix (supporting appendix)

Area	Technologies
Languages	Java 21; Python 3.11; TypeScript; Go; Terraform; Shell; SQL
Backend	Spring Boot; Spring Cloud; Spring Security; Spring Authorization Server; FastAPI; LangChain; LangGraph
AI/ML	RAG; embeddings; agentic AI; LLM guardrails; prompt security; ML evaluation; PCA; LBP; HOG; SURF; Harris
Data/Streaming	Apache Kafka; Kafka Streams; Redis; PostgreSQL; DuckDB; Parquet
Cloud/IaC	AWS; Terraform; Amazon EKS; Lambda; RDS; DynamoDB; Route 53; S3; CloudWatch
Kubernetes/GitOps	Helm; Argo CD; Argo Rollouts; KEDA; HPA/VPA; Network Policies
DevOps/Security	GitHub Actions; Docker; Trivy; Gripe; Syft; Gitleaks; Checkov; OWASP ZAP
Testing	JUnit; Mockito; ArchUnit; REST Assured; Testcontainers; Pact; Playwright; k6; JMH
Observability	OpenTelemetry; Jaeger; Prometheus; Grafana; Micrometer; structured logging
Frontend	React; TypeScript; optional React Native for later mobile delivery

55. Complete Git Repository Layout

```
universal-ticketing-platform/
  README.md
  LICENSE
  CODEOWNERS
  .github/
    workflows/
    ci.yml
    security.yml
    performance.yml
    release.yml
  docs/
    architecture/
    context.md
    container.md
    deployment.md
    provider-integration.md
    data-platform.md
  adr/
  api/
  security/
    threat-model.md
    ai-threat-model.md
  runbooks/
  diagrams/
  services/
    api-gateway/
    identity-service/
    tenant-service/
    catalog-service/
    ticket-type-service/
    provider-service/
    search-service/
    availability-service/
    inventory-service/
    reservation-service/
    booking-service/
    pricing-service/
    payment-service/
    refund-service/
    ticket-service/
    notification-service/
    promotion-service/
    trip-service/
    fraud-service/
    analytics-service/
  integrations/
    provider-sdk/
    bus/
    train/
    movie/
    event/
  ai/
    ai-gateway/
    assistant/
    agent/
    rag/
```

- ■■■ recommendation/
- ■■■ data-analyst/
- ■■■ guardrails/
- ■■■ evaluation/
- ml/
- ■■■ demand-forecasting/
- ■■■ fraud-detection/
- ■■■ cancellation-prediction/
- ■■■ recommendation/
- ■■■ anomaly-detection/
- data/
- ■■■ kafka/
- ■■■ schemas/
- ■■■ dbt/
- ■■■ duckdb/
- ■■■ parquet/
- frontend/
- ■■■ consumer-web/
- ■■■ admin-web/
- infrastructure/
- ■■■ docker/
- ■■■ compose/
- ■■■ helm/
- ■■■ argocd/
- ■■■ argorollouts/
- ■■■ k8s/
- ■■■ terraform/
- tests/
- ■■■ contract/
- ■■■ e2e/
- ■■■ load/
- ■■■ security/
- scripts/
- dev/
- test/
- release/

56. Architecture Graphs

System Context

User/Admin → React Web → API Gateway → Ticketing Services → Provider Gateway → External Providers
 ↓
 Kafka Event Bus
 ↓
 Analytics / AI / ML Platform

Provider Aggregation

Search → Provider Router → [Bus Provider A | Bus Provider B | Train Provider A | Cinema A] → Normalize → Deduplicate → Rank → Universal Offers

Booking Flow

Search → Availability → Hold → Payment Authorization → Provider Book → Confirm → Ticket Issue → Notification
 ■
 ■■■ timeout/failure → release/compensate → reconciliation

AI Agent

User → AI Gateway → Intent/Policy → Agent → [Search | Availability | Booking | Cancellation | Analytics]
 ■
 Guardrails + Authorization + Audit

AI Data Analyst

Question → Semantic Layer → SQL Generator → SQL Validator → Read-only Warehouse → Statistical Analysis → Chart + Explanation + Forecast

CI/CD

Commit → GitHub Actions → Test → SAST/Secrets/IaC → SBOM → Build Image → Scan → Integration/Contract/E2E → k6 → GitOps → Argo CD → ZAP → Argo Rollouts → Smoke → Rollback

Cloud

Route 53 → Edge/LB/API Gateway → EKS → Services → RDS/Redis/Kafka
 ■→ S3
 ■→ Lambda
 ■→ CloudWatch

Data

Services → Outbox → Kafka → Stream Processing → Parquet/S3/DuckDB or Warehouse → Semantic Layer → BI + AI Data Analyst

Security

Internet → Gateway/WAF/Rate Limit → AuthN/AuthZ → Service Network Policies → Service-to-Service Auth → Data Encryption → Audit/SIEM

Ticket Lifecycle

AVAILABLE → HELD → CONFIRMED
HELD → EXPIRED → AVAILABLE
CONFIRMED → CANCELLED → REFUND_PENDING → REFUNDED

57. Free/Offline API Strategy

Do not make external paid APIs a prerequisite. Build Provider Simulator services for bus, train, movie and event domains. Use configurable mock data and deterministic responses. For AI, use a local model runtime and local embeddings where practical; keep LLM/provider adapters behind an AI Gateway. External developer/sandbox APIs can be added later through adapters. Never commit API keys; use environment variables and secret managers.

58. Recommended Build Order

1. Create repository and engineering standards
2. Create Docker Compose local platform
3. Create identity/auth and gateway
4. Create configuration and provider framework
5. Create canonical search and offer normalization
6. Create inventory/seat/hold engine
7. Create booking/payment/ticket lifecycle
8. Create three mock providers
9. Create React consumer/admin applications
10. Add Kafka events and reconciliation
11. Add observability
12. Add CI/CD and security gates
13. Add AI assistant and RAG
14. Add AI agent with tool authorization
15. Add AI Data Analyst
16. Add ML models
17. Deploy to local Kubernetes
18. Add Helm/Argo CD/Argo Rollouts
19. Add Terraform/EKS AWS deployment
20. Perform performance, security and failure testing

59. Engineering Rules

- No provider-specific schema in core domain
- No direct LLM calls from business services
- No payment action without explicit authorization
- Every mutating API is idempotent where applicable
- Every external provider call has timeout and failure handling
- Every important business action emits an event
- Every service exposes health/readiness and metrics
- Every PR passes automated tests and security gates
- Every production change is GitOps-controlled
- Every configuration change is versioned and auditable
- Every AI tool has an explicit permission policy

- Every analytics query runs against read-only data